Root-Shooting MCTS: Best-Arm-First Simulation Allocation for Low-Budget Planning

Anonymous Authors[1]

Abstract

We study Monte Carlo Tree Search in the regime where each decision is made under a very small fixed simulation budget. Our hypothesis is that, in this regime, vanilla UCT often fails not because it explores too shallowly everywhere, but because it allocates simulations too diffusely across the tree before identifying the best root action. We therefore propose Root-Shooting MCTS (RS-MCTS), a simple root-focused variant that treats each decision as a fixed-budget best-arm identification problem over root actions. RS-MCTS allocates simulations in successive-halving-style rounds over root actions; each probe commits to one root action and then performs only shallow subtree search with heuristic completion inside that action's subtree. On deterministic synthetic planning tasks—grid reward collection, graph orienteering, and deadline task scheduling—RS-MCTS improves mean score, regret to best observed, and optimality gap relative to vanilla UCT on two of the three domains across all tested budgets $\{16, 64, 128\}$, with the largest gains at the smallest budget. In graph orienteering, however, the advantage is limited to smaller budgets and slightly reverses at budget 128, indicating that the method is better understood as a low-budget specialist than as a universal replacement for UCT. A rollout ablation further suggests that the main gains come from root-focused allocation itself, while rollout quality provides an additional but domain-dependent benefit.

1. Introduction

Monte Carlo Tree Search (MCTS) is a standard approach for online planning because it can improve decision quality by allocating simulation effort adaptively across a search tree (Kocsis & Szepesvari, 2006; Browne et al., 2012). In many practical settings, however, planning is not given enough budget for the asymptotic strengths of tree balancing to matter. Instead, a planner may only have a handful of simulations before it must commit to the next action. In that regime, the objective is less about building a globally balanced tree and more about choosing the best action now.

This distinction suggests a mismatch between low-budget planning and vanilla UCT. UCT inherits a cumulative-regret-style bandit exploration rule (Auer et al., 2002; Kocsis & Szepesvari, 2006); it is designed to distribute effort adaptively through the tree, not to directly optimize fixed-budget root decision quality. Prior work has argued that simple-regret or best-arm objectives are often more appropriate when one only cares about the final selected action (Tolpin & Shimony, 2012; Bubeck et al., 2011). This perspective is especially natural in receding-horizon planning with fixed per-decision budgets: if one wrong root action is chosen, later search cannot undo the mistake.

We investigate this issue in deterministic synthetic planning tasks with small per-decision budgets. These tasks are intentionally modest in size, but they stress a specific failure mode: many actions are superficially plausible, while only a few are competitive after limited lookahead. Our proposal, Root-Shooting MCTS (RS-MCTS), explicitly treats root action choice as a fixed-budget best-arm identification problem. Instead of growing a single shared tree under vanilla UCT, RS-MCTS allocates probes across root actions in elimination rounds inspired by successive halving (Karnin et al., 2013; Cazenave, 2015). Each probe commits to a root action, then performs only shallow subtree search plus rollout completion within that action's subtree. Poorly performing root actions are pruned, and later budget is concentrated on survivors.

The method is deliberately simple. We do not modify the vanilla baseline. We also do not claim a new theoretical guarantee. Our goal is empirical and diagnostic: to test whether changing the allocation objective at the root helps more than deeper generic tree growth when simulation budgets are very small. The answer is mostly yes, but not uniformly. On grid reward collection and deadline task scheduling,

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

RS-MCTS substantially outperforms vanilla UCT across all tested budgets. On graph orienteering, it helps at budget 16, is nearly tied at 64, and is slightly worse at 128. This negative result is important: it shows that root-focus is valuable in the intended regime, but can over-concentrate when larger budgets make broader tree balancing worthwhile.

Our contributions are:

- We propose RS-MCTS, a simple root-focused MCTS variant for low-budget per-decision planning that combines successive-halving-style root allocation with shallow subtree probes.
- We provide a controlled empirical study on deterministic planning benchmarks comparing RS-MCTS against unchanged vanilla UCT, random, greedy, and beam-search baselines.
- We show that RS-MCTS yields large gains over vanilla UCT on grid reward collection and deadline task scheduling, especially at budget 16, while revealing a meaningful crossover on graph orienteering at larger budgets.
- We include a rollout ablation showing that root-focused allocation is the primary source of improvement, with rollout quality contributing additional but environment-dependent gains.

2. Related Work

Our work sits at the intersection of MCTS, pure-exploration bandits, and simulation allocation for online planning.

Vanilla MCTS and UCT. MCTS became a general framework for simulation-based tree search through early work by Coulom and later UCT (Coulom, 2006; Kocsis & Szepesvari, 2006), with many practical enhancements surveyed by Browne et al. (Browne et al., 2012). Standard UCT applies upper-confidence exploration recursively in the tree, balancing exploitation and exploration at every interior node. This behavior is often desirable asymptotically, but it does not directly optimize the finite-budget quality of the final root action. Coquelin and Munos (Coquelin & Munos, 2007) further analyzed tree-bandit methods and highlighted the subtle tradeoffs created by recursive optimistic search.

Simple regret and best-arm identification. A central motivation for our method is that low-budget planning is closer to a pure-exploration problem than a cumulative-regret one. In stochastic bandits, this distinction is formalized by simple-regret and best-arm identification objectives (Bubeck et al., 2011; Audibert et al., 2010; Barrier et al., 2022). UCB-style rules derive from cumulative-regret analysis (Auer et al., 2002), whereas fixed-budget pure-exploration methods allocate samples to maximize final arm-selection accuracy. Tolpin and Shimony (Tolpin & Shimony, 2012) explicitly connected this distinction to MCTS, arguing that simple-regret criteria can be more appropriate when the planner only cares about the eventual move choice. Kaufmann and Koolen (Kaufmann & Koolen, 2017) likewise studied root action selection in MCTS through a best-arm identification lens.

Successive elimination and successive halving. Fixed-budget best-arm methods such as successive rejects and successive halving (Audibert et al., 2010; Karnin et al., 2013) inspire our root allocation schedule. We do not claim novelty in the elimination idea itself. Rather, we use it as a simple design principle for small-budget planning: spend early budget broadly over root actions, eliminate clear losers, and spend later budget on survivors. This differs from vanilla UCT, which expands and revisits interior nodes under a single tree objective. Kalyanakrishnan et al. (Kalyanakrishnan et al., 2012) also provide relevant confidence-based pure-exploration perspectives.

Sequential halving inside tree search. The closest prior line is sequential-halving-based MCTS. Cazenave (Cazenave, 2015) applied sequential halving to trees, and Sagers et al. (Sagers et al., 2024) developed an anytime follow-up. Our work differs in emphasis. We target a fixed per-decision budget benchmark in deterministic synthetic planning; we keep the method simple and local to the root; and we study whether shallow subtree probes are enough to improve the immediate action choice under severe budget limits. In spirit, our method is closer to a practical root-focused allocation rule than to a full replacement for general-purpose tree search.

Root-focused and flat Monte Carlo planning. Flat or shallow-root simulation methods have long been considered attractive when deep shared trees are too expensive (Teytaud & Flory, 2011). More broadly, simulation-based planning under limited budget has a long history predating MCTS, including sparse sampling and adaptive sampling methods (Kearns et al., 1999; Chang et al., 2005). Recent planning work has also revisited root best-arm ideas in modern settings such as task planning in robotics (Jin et al., 2023). Our contribution is not to introduce root selection as a concept, but to show that a particularly simple root-shooting formulation is effective in the low-budget deterministic benchmark regime considered here.

3. Background

We consider finite-horizon deterministic planning problems. A problem provides an initial state s_0, a set of legal actions

$\mathcal{A}(s)$ at each state, a deterministic transition function $s' = T(s, a)$, a terminal predicate, and a scalar state evaluation $V(s)$ equal to the cumulative score achieved so far. At each decision step, the planner receives a fixed simulation budget B and must choose one action, after which planning restarts from the next state.

Vanilla UCT. Given root state s, vanilla UCT constructs a tree incrementally. Each simulation traverses the tree using an upper-confidence rule,

$$\mathrm{UCT}(c) = \hat{Q}(c) + C\sqrt{\frac{\log N(p)}{N(c)}},$$

where p is the parent node, $N(\cdot)$ denotes visits, and $\hat{Q}(c)$ is the empirical mean rollout value of child c (Kocsis & Szepesvari, 2006). Once a node with untried actions is reached, one action is expanded, followed by a rollout to terminal and a Monte Carlo backup. After B simulations, the root action is chosen from the resulting child statistics.

Under large budgets, this procedure can be effective because it both distinguishes root actions and improves deeper value estimates. Under very small budgets, however, the planner may spend a large fraction of simulations expanding or revisiting interior nodes before it has confidently identified the best root action.

Root action selection as fixed-budget best-arm identification. If the planning problem is receding-horizon and the immediate output is only the next action, root action selection can be viewed as a K-armed fixed-budget identification problem, with $K = |\mathcal{A}(s)|$. Each arm corresponds to a root action a, and a probe of arm a estimates the downstream return from first taking a and then planning within the resulting subtree. In this framing, the goal is to minimize a simple-regret-like error at the root rather than to maximize cumulative gain from all intermediate simulations (Bubeck et al., 2011; Tolpin & Shimony, 2012).

This motivates an elimination strategy:

1. sample all root actions enough to get coarse estimates;
2. eliminate clearly inferior actions;
3. concentrate the remaining budget on the surviving root actions.

RS-MCTS instantiates exactly this logic while still using MCTS-style local search inside each probed subtree.

4. Method

We now describe Root-Shooting MCTS (RS-MCTS). The method is designed for a single planning decision at root state s with legal actions $A = \{a_1, \ldots, a_K\}$ and simulation budget B.

4.1. Root-shooting allocation

RS-MCTS maintains a survivor set $S \subseteq A$, initialized as $S = A$. Budget is allocated in rounds. In each round, every surviving action receives the same number of probes; after the round, the bottom half of actions are eliminated according to a lower-confidence ranking. This is similar in spirit to successive halving (Karnin et al., 2013), but our objective is not a generic tree-search replacement. Instead, the elimination happens only at the root, and each probe uses shallow subtree search inside one chosen root action.

Following the implementation used in our experiments, if B_{rem} is the remaining budget and $|S|$ the number of survivors, the number of probes per surviving arm in the current round is

$$m = \max\left(1, \left\lfloor \frac{B_{\mathrm{rem}}}{|S| \cdot R_{\mathrm{left}}} \right\rfloor\right),$$

where $R_{\mathrm{left}} = \lceil \log_2 |S| \rceil$ is the approximate number of elimination rounds left. This rule is simple and ensures that each round remains affordable while preserving enough budget for later concentration.

For each action $a \in S$, RS-MCTS stores empirical probe count $n(a)$ and sum of probe returns $\Sigma(a)$, giving mean estimate

$$Q(a) = \frac{\Sigma(a)}{\max(n(a), 1)}.$$

After a round, actions are ranked by a lower-confidence score

$$\mathrm{LCB}(a) = Q(a) - \sqrt{\frac{2\log(\max(B_{\mathrm{used}}, 2))}{\max(n(a), 1)}}.$$

The top half under this ranking survive to the next round.

4.2. Shallow subtree probes

A probe of root action a proceeds in two stages:

1. commit to a and deterministically transition to child state $s' = T(s, a)$;
2. run a small internal planner only inside the subtree rooted at s'.

The internal planner is intentionally shallow. In our implementation, it builds a tiny UCT tree inside the chosen subtree using a reduced exploration constant and a small depth-dependent internal budget. If the probe does not reach terminal, it completes the simulation with a rollout policy. This preserves a key strength of MCTS—local adaptation

Algorithm 1: Root-Shooting MCTS at root state s with budget B

Input: root actions A, probe routine $\mathrm{Probe}(s, a)$
Initialize: $S \leftarrow A$; $n(a) \leftarrow 0$; $\Sigma(a) \leftarrow 0$ for all $a \in A$
while $|S| > 1$ and budget remains **do**
  choose probes-per-arm m from remaining budget
  for each $a \in S$ **do**
    repeat m times:
      $v \leftarrow \mathrm{Probe}(s, a)$
      $n(a) \leftarrow n(a) + 1$, $\Sigma(a) \leftarrow \Sigma(a) + v$
  compute $Q(a) = \Sigma(a)/n(a)$ and $\mathrm{LCB}(a)$ for $a \in S$
  keep top half of S by $\mathrm{LCB}(a)$
end while
use any leftover budget on the remaining survivors
return $\arg\max_{a \in A} Q(a)$ among sampled actions

Figure 1. RS-MCTS treats root action choice as fixed-budget best-arm identification, while each probe performs only shallow local search after committing to the chosen root action.

inside promising subtrees—without paying the cost of globally expanding many root alternatives and many interior branches at once.

The resulting probe value is the terminal score obtained from that committed simulation. Because the benchmark domains are deterministic, repeated probes of the same root action mainly capture differences in subtree search behavior rather than transition noise. This is useful for root discrimination, but also means our conclusions are about deterministic low-budget planning rather than stochastic robustness.

4.3. Rollout policy and ablation

The full RS-MCTS variant uses a heuristic rollout with probability 0.85 of taking the problem's provided greedy action and probability 0.15 of taking a random legal action. This keeps the rollout simple while injecting some stochasticity. To isolate whether gains come from root allocation rather than only from stronger rollouts, we also evaluate a rollout ablation that is identical to RS-MCTS in root allocation and subtree probing but replaces the heuristic rollout with a purely random rollout.

Discussion. RS-MCTS changes the objective of simulation allocation rather than merely retuning UCT. Tuning the UCT constant changes how optimistic tree search is, but still optimizes a single recursive tree-wide exploration rule. In contrast, RS-MCTS explicitly privileges root identification under scarce budget. This should help when the main failure mode is sampling too many interior nodes before the root choice is reliable. It can hurt when deeper shared structure is important and enough budget is available to exploit it, which we indeed observe on graph orienteering at budget 128.

5. Experimental Setup

Environments. We evaluate on three deterministic synthetic planning benchmarks provided by the fixed harness:

- **Grid reward collection:** navigate a 7×7 grid with blocked cells, step penalties, and collectible rewards under a step budget.
- **Graph orienteering:** visit valuable graph nodes under a travel-time budget.
- **Deadline task scheduling:** assign tasks with durations, deadlines, priorities, and precedence constraints to machines, where early sequencing mistakes can create large downstream penalties.

Each environment contains 6 instances, for 18 planning problems total. Exact optimal scores are available for grid reward collection and graph orienteering through dynamic programming in the benchmark code; scheduling does not provide an exact optimum.

Budgets and planners. We use the benchmark's fixed per-decision budgets $\{16, 64, 128\}$ for MCTS-based planners. Baselines are unchanged vanilla UCT MCTS, random, greedy, and beam search with width 4. The main proposed planner is RS-MCTS. For ablation, we additionally evaluate an otherwise identical RS-MCTS variant with random rollouts.

Metrics. We report mean score, regret to best observed, and optimality gap when available. We also use matched scenario-level comparisons between proposed and vanilla planners to compute score deltas, regret deltas, and win rates.

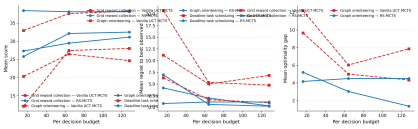
Implementation details. The vanilla UCT baseline is left unchanged exactly as required by the benchmark. RS-MCTS uses a shallow subtree exploration constant of 0.9, heuristic rollout probability 0.85, and a subtree depth limit that increases mildly with budget: 2 for budget 16, 3 for budget 64, and 4 for budget 128. These settings were taken from the successful discovered implementation and were not further tuned in the paper phase.

6. Experiments

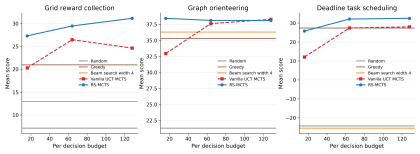
6.1. Main results

Figures 2 and 3 summarize the main comparison between RS-MCTS and vanilla UCT. The results support the central hypothesis on two of the three environments.

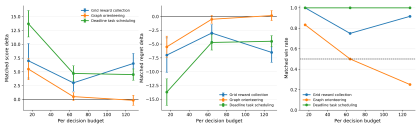
Table 1 reports the policy-level averages used in the main paper.



(a) Score, regret, and optimality gap overview.



(b) Score curves with non-MCTS baselines.



(c) Matched proposed-vs-vanilla deltas.

Figure 2. Main comparison figures. Across the tested budgets, RS-MCTS consistently improves over vanilla UCT on grid reward collection and deadline task scheduling, while graph orienteering shows a low-budget gain but a slight reversal at budget 128.

Table 1. Main results for vanilla UCT and RS-MCTS. Higher score is better; lower regret and lower optimality gap are better. Scheduling has no exact optimality gap in the benchmark.

Environment	Planner	Budget	Mean score	Mean regret	Mean optimality gap
Grid reward collection	Vanilla UCT	16	20.293	11.172	12.192
	RS-MCTS	16	**27.293**	**4.172**	**5.192**
	Vanilla UCT	64	26.460	5.005	6.025
	RS-MCTS	64	**29.460**	**2.005**	**3.025**
	Vanilla UCT	128	24.627	6.838	7.858
	RS-MCTS	128	**31.132**	**0.333**	**1.353**
Graph orienteering	Vanilla UCT	16	32.953	6.337	9.657
	RS-MCTS	16	**38.450**	**0.840**	**4.160**
	Vanilla UCT	64	37.620	1.670	4.990
	RS-MCTS	64	**38.123**	**1.167**	**4.487**
	Vanilla UCT	128	**38.287**	**1.003**	**4.323**
	RS-MCTS	128	38.120	1.170	4.490
Deadline task scheduling	Vanilla UCT	16	12.075	20.698	—
	RS-MCTS	16	**25.783**	**6.990**	—
	Vanilla UCT	64	27.417	5.356	—
	RS-MCTS	64	**32.125**	**0.648**	—
	Vanilla UCT	128	28.016	4.757	—
	RS-MCTS	128	**32.508**	**0.265**	—

Grid reward collection. This is the clearest success case for RS-MCTS. It beats vanilla UCT at all three budgets by score, regret, and optimality gap. The score advantage is +7.0 at budget 16, +3.0 at budget 64, and +6.505 at budget 128. Notably, vanilla UCT improves from 16 to 64 but drops at 128, while RS-MCTS improves monotonically. This is consistent with the claim that extra vanilla simulations can still be spent inefficiently inside the tree, whereas RS-MCTS uses additional budget to further refine the root choice.

Deadline task scheduling. This is the second strong success case. RS-MCTS again wins at every budget, with especially large gains at budget 16: mean score improves from 12.075 to 25.783, and mean regret drops from 20.698 to 6.990. The gap narrows but remains substantial at budgets 64 and 128. This domain matches our motivating intuition well: a poor early root choice can create large downstream penalties, so reliable root identification matters more than symmetric deeper exploration.

Graph orienteering. This is the main caveat. RS-MCTS improves over vanilla at budget 16 and is still slightly better at 64, but vanilla slightly overtakes it at 128. The difference at 128 is small (-0.167 score for RS-MCTS relative to vanilla), yet directionally important. This suggests that the root-focused allocation is most useful when budget is scarce, but may over-concentrate too early once enough simulations are available for deeper tree balancing to become useful.

Matched comparisons. Figure 2(c) strengthens the above interpretation by comparing planners instance-by-instance. On grid and scheduling, RS-MCTS has positive matched

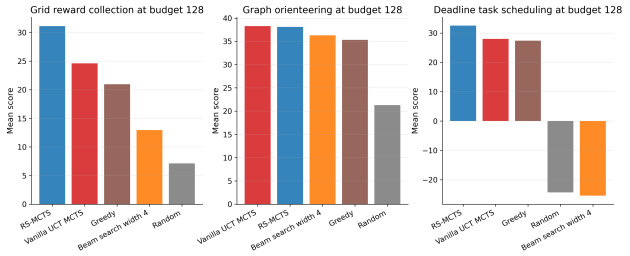


Figure 3. Environment-wise comparison at the largest available budget for budgeted methods. RS-MCTS is best on grid reward collection and deadline task scheduling, while graph orienteering is essentially tied between RS-MCTS and vanilla UCT.

score deltas and negative matched regret deltas at every budget, with the largest advantage at budget 16. On graph orienteering, the matched score advantage decreases with budget and becomes slightly negative at 128, mirroring the aggregate result.

6.2. Comparison to non-MCTS baselines

Figures 2(b) and 3 compares RS-MCTS to random, greedy, and beam search. RS-MCTS is clearly above random and beam search in all environments. It also outperforms greedy in grid reward collection and deadline task scheduling. In graph orienteering, however, greedy and beam search are already fairly strong, suggesting less room for root-focused search to improve over simpler decision rules.

6.3. Rollout ablation

To test whether gains come mainly from root allocation or from rollout strength, we compare RS-MCTS to an otherwise identical variant that replaces the heuristic rollout with a purely random rollout.

The ablation results are mixed but informative:

- On **deadline task scheduling**, the ordering is consistently RS-MCTS $>$ random-rollout ablation $>$ vanilla UCT. This suggests the root-focused allocation is the main improvement, while heuristic rollout yields an additional gain.
- On **grid reward collection**, the same ordering holds for score at all budgets. Thus root allocation explains a large part of the benefit, but rollout quality remains useful.
- On **graph orienteering**, the picture differs. The random-rollout ablation slightly exceeds full RS-MCTS at budget 64 and is competitive at 128. Thus the root-focused allocation still appears helpful at small budgets, but the specific rollout choice is not universally beneficial in this domain.

This is a useful negative result. It argues against an overly strong claim that our full planner dominates because of a generally superior rollout policy. Instead, the main mechanism is root-focused allocation, while the best rollout design is environment-dependent.

6.4. What the results do and do not show

The evidence supports a targeted claim: RS-MCTS is effective when per-decision budgets are small and the main challenge is identifying the best root action quickly. It does not support the stronger claim that RS-MCTS is a universal improvement over vanilla UCT at all budgets or in all domains. Graph orienteering demonstrates that deeper balanced search can recover and slightly surpass the root-focused method when budget becomes less restrictive.

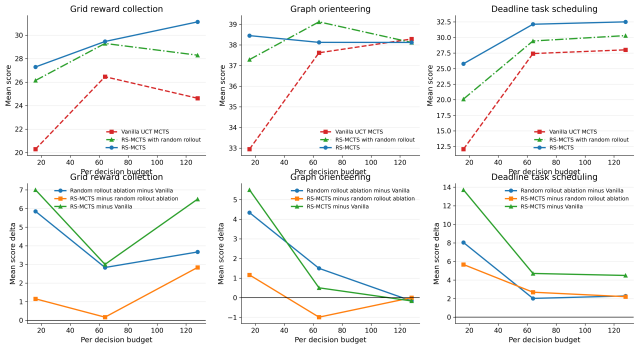
Because the environments are deterministic, another limitation is that some of the gains may come from lower variance and more stable value concentration rather than from a universally better exploration principle. Extending the study to stochastic domains would be necessary before making broader claims about robustness.

7. Conclusion

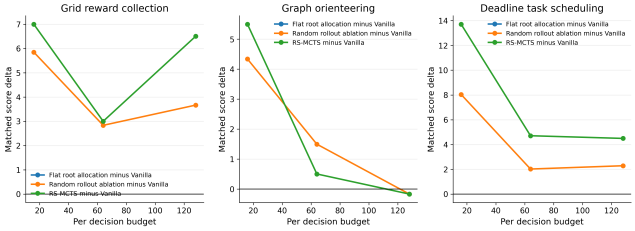
We introduced Root-Shooting MCTS, a simple MCTS variant that treats each planning step as a fixed-budget best-arm identification problem over root actions. The method allocates simulation effort in successive-halving-style rounds at the root and uses only shallow subtree search after committing to each candidate action. This shifts the search objective from generic tree-wide exploration toward immediate root decision quality.

Empirically, the method works well in the intended regime. On deterministic grid reward collection and deadline task scheduling, RS-MCTS consistently improves over unchanged vanilla UCT across all tested budgets, with the largest gains at budget 16. It also improves regret and optimality gap, supporting the claim that the core benefit is better root-action identification rather than merely higher rollout variance reduction. On graph orienteering, however, the benefit is limited to lower budgets and slightly reverses at budget 128. We view this as an important result, not a failure to report: it shows that the method is best understood as a low-budget specialist rather than a universal replacement for UCT.

The rollout ablation further suggests that root-focused allocation is the main source of improvement, while rollout design provides an additional but domain-dependent contribution. A natural next step is therefore an adaptive hybrid



(a) Absolute score comparison.



(b) Matched deltas versus vanilla across variants.

Figure 4. Rollout ablation. The random-rollout ablation keeps the same root allocation mechanism as RS-MCTS but weakens the rollout policy.

that behaves like RS-MCTS at very low budgets but relaxes toward broader UCT-style tree balancing once budget becomes large enough that deeper shared structure matters.

Impact Statement

This paper studies low-budget planning algorithms in small deterministic synthetic benchmarks. The work is methodological and does not introduce a directly deployed system. Potential positive impact includes improving the reliability of compute-constrained planners. Potential negative impact is limited in the current setting, though any improvement to automated planning could eventually be incorporated into higher-stakes autonomous systems. Our empirical claims are intentionally narrow: the method is only evaluated on deterministic synthetic tasks and should not be assumed to generalize to stochastic or safety-critical domains without further study.

References

Audibert, J.-Y., Bubeck, S., and Munos, R. Best arm identification in multi-armed bandits. pp. 41–53, 2010.

Auer, P., Cesa-Bianchi, N., and Fischer, P. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47:235–256, 2002.

Barrier, A., Garivier, A., and Stoltz, G. On best-arm identification with a fixed budget in non-parametric multi-armed bandits. *ArXiv*, abs/2210.00895, 2022.

Browne, C., Powley, E., Whitehouse, D., Lucas, S., Cowling, P., Rohlfshagen, P., Tavener, S., Liebana, D. P., Samothrakis, S., and Colton, S. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4:1–43, 2012.

Bubeck, S., Munos, R., and Stoltz, G. Pure exploration in finitely-armed and continuous-armed bandits. *Theor. Comput. Sci.*, 412:1832–1852, 2011.

Cazenave, T. Sequential halving applied to trees. *IEEE Transactions on Computational Intelligence and AI in Games*, 7:102–105, 2015.

Chang, H., Fu, M., Hu, J., and Marcus, S. An adaptive sampling algorithm for solving markov decision processes. *Oper. Res.*, 53:126–139, 2005.

Coquelin, P.-A. and Munos, R. Bandit algorithms for tree search. *ArXiv*, abs/cs/0703062, 2007.

Coulom, R. Efficient selectivity and backup operators in monte-carlo tree search. pp. 72–83, 2006.

Jin, D., Park, J., and Lee, K. Perturbation-based best arm identification for efficient task planning with monte-carlo tree search. *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5758–5764, 2023.

Kalyanakrishnan, S., Tewari, A., Auer, P., and Stone, P. Pac subset selection in stochastic multi-armed bandits. pp. 227–234, 2012.

Karnin, Z. S., Koren, T., and Somekh, O. Almost optimal exploration in multi-armed bandits. pp. 1238–1246, 2013.

Kaufmann, E. and Koolen, W. M. Monte-carlo tree search by best arm identification. *ArXiv*, abs/1706.02986, 2017.

Kearns, M., Mansour, Y., and Ng, A. A sparse sampling algorithm for near-optimal planning in large markov decision processes. *Machine Learning*, 49:193–208, 1999.

Kocsis, L. and Szepesvari, C. Bandit based monte-carlo planning. pp. 282–293, 2006.

Sagers, D., Winands, M., and Soemers, D. J. Anytime sequential halving in monte-carlo tree search. *ArXiv*, abs/2411.07171, 2024.

Teytaud, O. and Flory, S. Upper confidence trees with short term partial information. pp. 153–162, 2011.

Tolpin, D. and Shimony, S. E. Mcts based on simple regret. *ArXiv*, abs/1207.5536, 2012.

Supplementary Material

A. Additional discussion of the benchmarked method

The discovered implementation of RS-MCTS used the following practical choices:

- root elimination by lower-confidence ranking;
- shallow subtree UCT with a reduced exploration constant;
- a heuristic rollout that chooses the benchmark's greedy action with probability 0.85 and a random action otherwise;
- subtree depth limit 2, 3, or 4 for budgets 16, 64, and 128 respectively.

We retained these settings exactly for the reported experiments instead of retuning them further, to keep the study faithful to the discovered method and benchmark outputs.

B. Additional ablation family figures

The full ablation run also included a stronger family comparison with an additional flat root-allocation variant. We did not rely on that variant in the main claims because the main benchmark summary for RS-MCTS versus vanilla UCT already answers the paper's central question, but those figures are useful for qualitative interpretation.

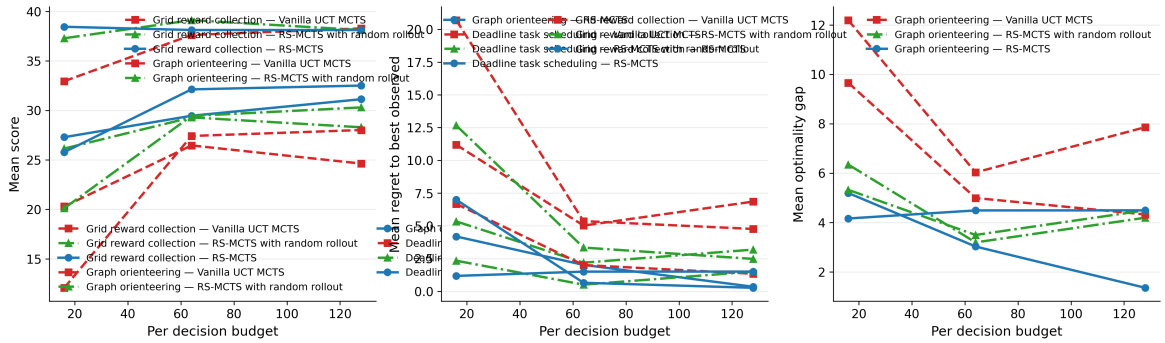


Figure 5. Extended ablation-family overview across score, regret, and optimality gap. This figure includes vanilla UCT, flat root allocation MCTS, the random-rollout ablation, and full RS-MCTS.

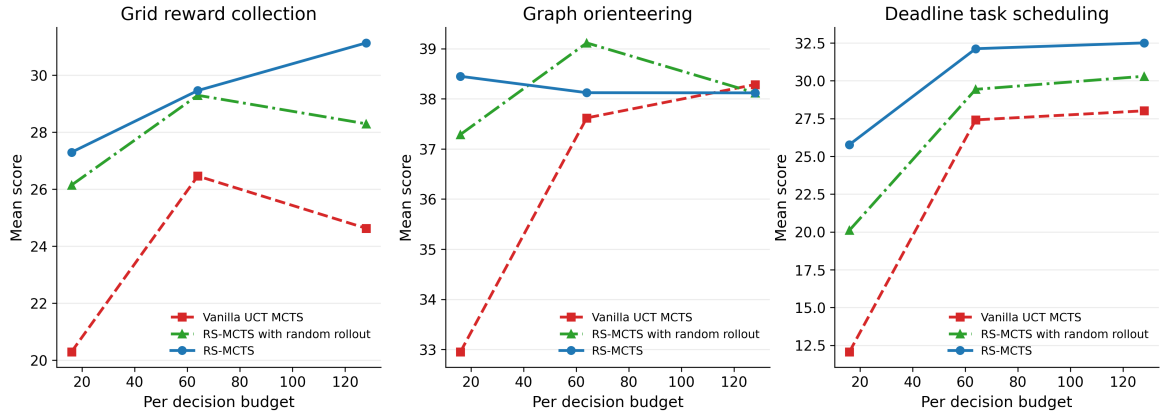


Figure 6. Extended ablation-family score curves by environment. These plots further support the main conclusion that root-focused allocation is most beneficial in low-budget grid and scheduling tasks.

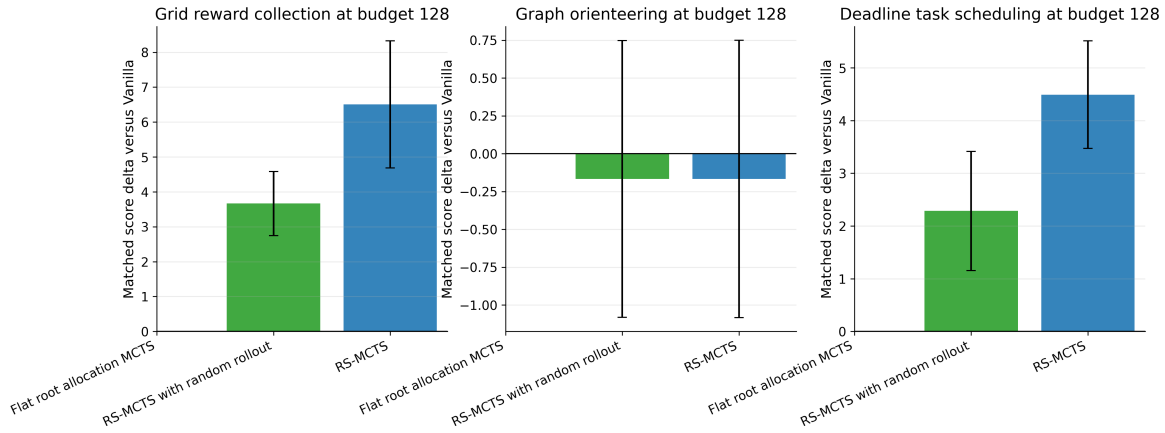


Figure 7. Matched score deltas versus vanilla at the largest budget for multiple root-focused variants. The figure highlights that graph orienteering is the main environment where variant ranking is unstable.

C. Limitations and future work

Our study has several limitations. First, all tasks are deterministic. This makes the benchmark appropriate for testing root allocation under small budgets, but it leaves open whether the same advantage appears in stochastic domains where probes have higher variance. Second, budgets were limited to $\{16, 64, 128\}$. This is enough to reveal the intended low-budget regime and a crossover on graph orienteering, but not enough to characterize asymptotic behavior. Third, the method uses hard elimination at the root. While effective in two domains, this can plausibly discard good actions too early in domains where deeper information matters more.

A natural future direction is an adaptive version of RS-MCTS that uses root elimination only when budget is below a problem-dependent threshold, or decays toward ordinary UCT as confidence accumulates. Another direction is to test whether the same root-focused logic remains helpful in stochastic planning, partially observable settings, or larger combinatorial problems.

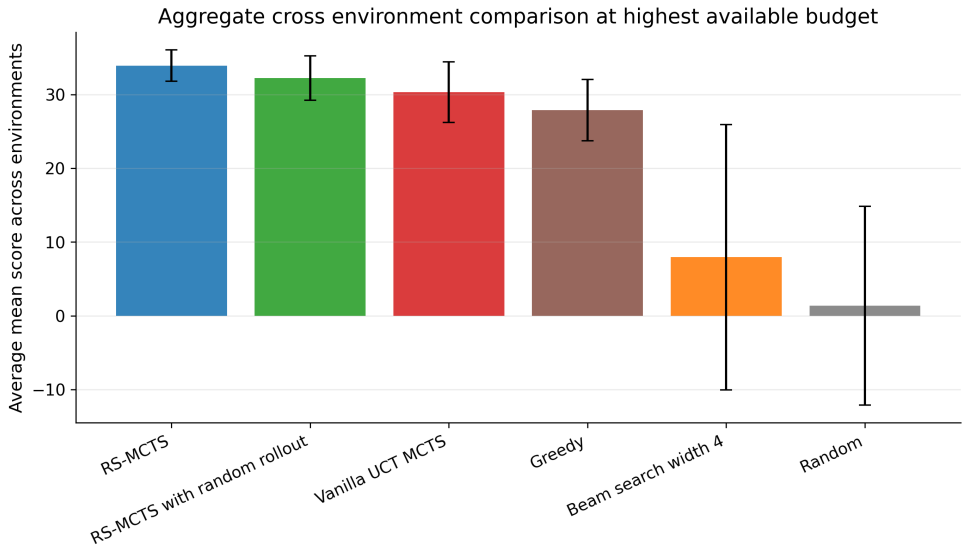


Figure 8. Aggregate cross-environment ranking at highest available budget from the final figure aggregator. This summary is convenient but should be interpreted together with the environment-specific plots in the main paper.